

Практична робота 4

Тема: MongoDB та Mongoose: міграція REST API на базу даних

Мета: набуття практичних навичок роботи з MongoDB та Mongoose шляхом міграції REST API з лабораторної роботи №3 з in-memory сховища на повноцінну базу даних

Хід роботи: Варіант 5

Завдання 1.

MongoDB Atlas та MongoDB Compass

Зареєструйтесь у хмарному сервісі MongoDB та підключіться до нього через GUIклієнт.

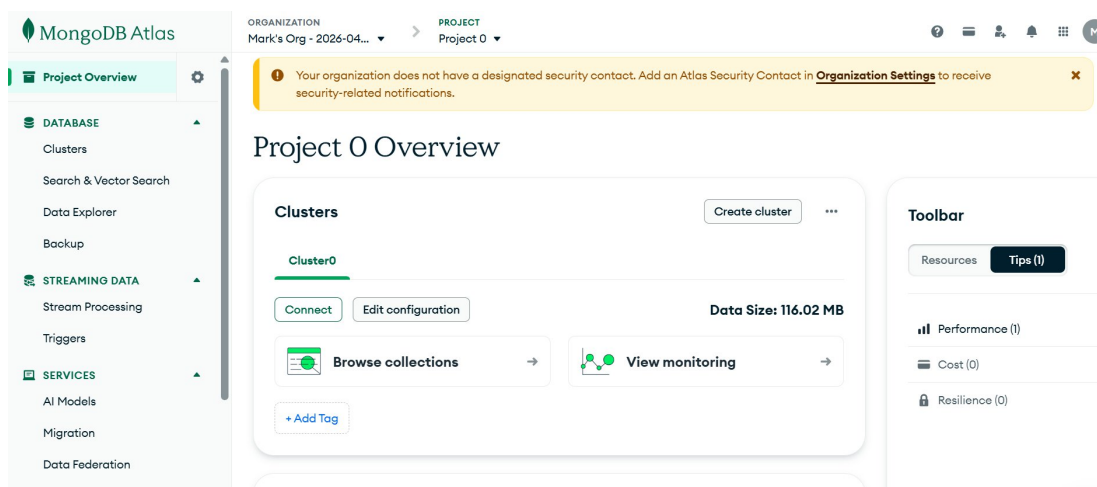


Рис. 4.1 – Mongo Atlas

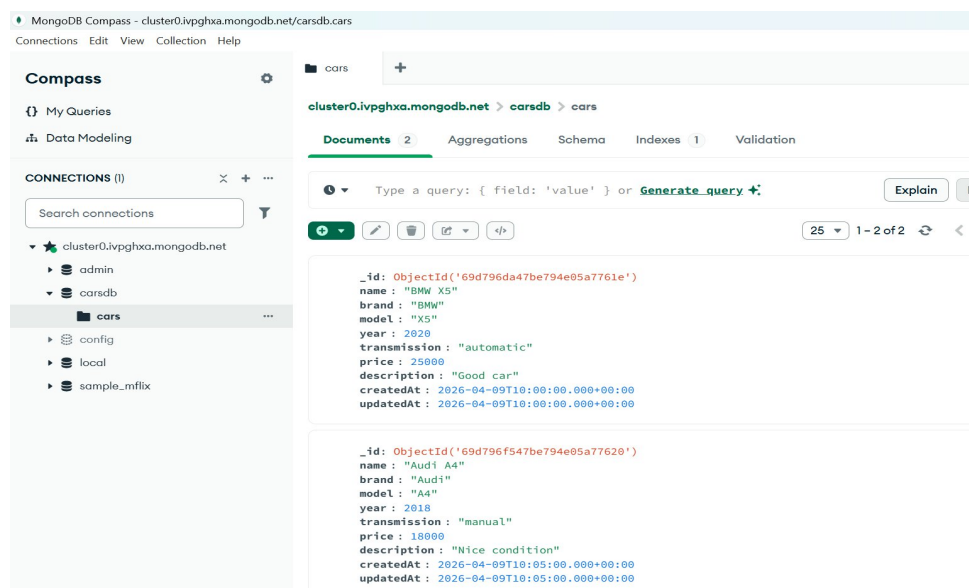


Рис. 4.2 – Mongo Compass

		Ліщинський В. В.			ДУ «Житомирська політехніка». 24.121.05.000 - ПЗ	Арк.
		Петросян А. Р.				1
Змн.	Арк.	№ докум.	Підпис	Дата		

Завдання 2

Підключення Mongoose та оновлення server.ts Встановіть Mongoose та налаштуйте підключення до бази даних перед стартом сервера.

Вимоги:

- Встановіть mongoose
- Встановіть dotenv та підключіть його на початку server.ts.
- Створіть src/config/database.ts з асинхронною функцією підключення до бази даних.

```
src > config > TS database.ts > mongoose.connection.on('disconnected') callback
1  import mongoose from 'mongoose';
2
3  export async function connectDB(): Promise<void> {
4      const uri = process.env.MONGODB_URI;
5      if(!uri) {
6          throw new Error('MONGODB_URI is not defined in environment variables');
7      }
8
9      await mongoose.connect(uri);
10     console.log('MongoDB підключено');
11 }
12
13 mongoose.connection.on('error', (err) => {
14     console.error('MongoDB помилка:', err);
15 });
16
17 mongoose.connection.on('disconnected', () => {
18     console.warn('MongoDB відключено');
19 });
```

Рис. 4.3 – Файл database.ts

		Ліщинський В. В.			ДУ «Житомирська політехніка». 24.121.05.000 - ПЗ	Арк.
		Петросян А. Р.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

src > TS server.ts > ...
1  import app from './app';
2  import { connectDB } from './config/database';
3  import mongoose from 'mongoose';
4  import dotenv from 'dotenv';
5
6  dotenv.config();
7
8  const PORT = process.env.PORT || 3000;
9
10 async function start() {
11   try {
12     await connectDB();
13
14     const server = app.listen(PORT, () => {
15       console.log(`Server running on port ${PORT}`);
16     });
17

```

Рис. 4.4 – Підключення env в server.ts

Завдання 3

Mongoose-схема та модель Створіть Mongoose-схему для вашої сутності, яка відображає ті самі атрибути, що й Zod-схема

Вимоги:

- Створіть src/models/{entity}.model.ts.
- Оголосіть TypeScript-інтерфейс для вашої сутності з усіма її полями (без `_id` — Mongoose додає його автоматично).
- Опишіть Mongoose-схему на основі цього інтерфейсу з увімкненим автоматичним управлінням датами (timestamps: true). Для кожного поля вкажіть тип, обов'язковість, перелік дозволених значень, дефолтне значення та обмеження на довжину/розмір тощо — відповідно до Zod-схеми з лаб. 3.
- Zod-схеми залишаються незмінними — вони валідують HTTP-запити, Mongoose-схема забезпечує цілісність у БД.
- Реалізуйте щонайменше 1 кастомний валідатор для одного з полів — функція-предикат з описовим повідомленням про помилку.
- Реалізуйте щонайменше 1 віртуальну властивість — computed поле, що не зберігається в БД, але обчислюється на основі наявних полів. Увімкніть включення віртуальних властивостей у JSON-серіалізацію, щоб вони потрапляли до API-відповідей

```

src > models > TS car.model.ts > [?] carSchema
1  import { Schema, model, HydratedDocument } from 'mongoose';
2
3  const currentYear = new Date().getFullYear();
4
5  export interface Car {
6      name: string;
7      brand: string;
8      model: string;
9      year: number;
10     transmission: 'manual' | 'automatic';
11     price: number;
12     description?: string;
13     createdAt: Date;
14     updatedAt: Date;
15 }
16
17 export type CarDocument = HydratedDocument<Car>;
18
19 const carSchema = new Schema<Car>(
20     {
21         name: {
22             type: String,
23             required: [true, 'Name is required'],
24             trim: true,
25             minlength: [1, 'Name must be at least 1 character long'],

```

Рис. 4.5 – Файл car.model.ts

Завдання 4

Заміна сховища на Mongoose Замініть Map-сховище з лаб. 3 на виклики Mongoose Model, зберігши той самий API-контракт.

Вимоги:

- Замініть весь вміст storage-модуля (src/storage/{entity}.ts) на функції, що використовують EntityModel.
- MongoDB генерує _id (ObjectId) автоматично — ручна генерація ідентифікатора та конструювання об'єкта більше не потрібні.
- Кожна операція реалізується відповідним методом моделі. При оновленні переконайтесь, що метод повертає оновлений документ і запускає валідатори схеми.
- Збережіть (або розширте за необхідності) ті самі статус-коди відповідей: 200, 201, 204, 400, 404.
- Функцію скидання сховища (reset) можна видалити — в тестах використовуватиметься очищення через модель

		Ліщинський В. В.			ДУ «Житомирська політехніка». 24.121.05.000 - ПЗ	Арк.
		Петросян А. Р.				4
Змн.	Арк.	№ докум.	Підпис	Дата		

```

src > storage > TS car.ts > updateCar
1  import { CarModel } from '../models/car.model';
2  import { CreateCarDTO, UpdateCarDTO } from '../schemas/car.schema';
3
4  type CarFilters = {
5    brand?: string;
6    transmission?: 'manual' | 'automatic';
7  };
8
9  type GetAllCarsOptions = {
10   filters?: CarFilters;
11   sort?: string;
12   page?: number;
13   limit?: number;
14 };
15
16 function buildCreatePayload(data: CreateCarDTO) {
17   const payload: {
18     name: string;
19     brand: string;
20     model: string;
21     year: number;
22     transmission: 'manual' | 'automatic';
23     price: number;
24     description?: string;
25   } = {

```

		Ліщинський В. В.			ДУ «Житомирська політехніка».24.121.05.000 - ПЗ	Арк.
		Петросян А. Р.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

41  function buildUpdatePayload(data: UpdateCarDTO) {
42      const payload: {
43          name?: string;
44          brand?: string;
45          model?: string;
46          year?: number;
47          transmission?: 'manual' | 'automatic';
48          price?: number;
49          description?: string;
50      } = {};
51
52      if (data.name !== undefined) {
53          payload.name = data.name;
54      }
55
56      if (data.brand !== undefined) {
57          payload.brand = data.brand;
58      }
59
60      if (data.model !== undefined) {
61          payload.model = data.model;
62      }
63
64      if (data.year !== undefined) {
65          payload.year = data.year;

```

Рис. 4.6 – Реалізація storage

Завдання 5

Розширте errorHandler для обробки специфічних помилок Mongoose та MongoDB.

Вимоги:

- Розширте src/middleware/errorHandler.ts, щоб правильно обробляти специфічні помилки Mongoose та MongoDB з відповідними статус-кодами. Обробку помилок Zod з лаб. 3 збережіть.
- Переконайтеся, що запит до неіснуючого, але валідного ObjectId повертає 404, а запит з невалідним форматом ID — 400


```

src > middleware > TS errorHandler.ts > errorHandler
1  import { ZodError } from 'zod';
2  import mongoose from 'mongoose';
3  import { MongoServerError } from 'mongodb';
4  import { Request, Response, NextFunction } from 'express';
5
6  export function errorHandler(
7      err: Error,
8      req: Request,
9      res: Response,
10     next: NextFunction
11 ) {
12     if (err instanceof ZodError) {
13         return res.status(400).json({
14             errors: err.issues.map(i => ({
15                 field: i.path.join('.'),
16                 message: i.message,
17             })),
18         });
19     }
20
21     if (err instanceof mongoose.Error.CastError) {
22         return res.status(400).json({
23             message: `Invalid ${err.path}`,
24         });
25     }
26
27     if (err instanceof mongoose.Error.ValidationError) {
28         return res.status(400).json({
29             errors: Object.values(err.errors).map((error) => ({
30                 field: error.path,
31                 message: error.message,
32             })),
33         });
34     }
35
36     if (err instanceof MongoServerError && err.code === 11000) {
37         return res.status(409).json({
38             message: 'Duplicate key error',
39             keyValue: err.keyValue,
40         });
41     }
42
43     console.error('Unexpected error:', err);
44     res.status(500).json({ error: 'Internal server error', status: 500 });
45 }

```

Рис. 4.7 – Розширено перехоплення помилок

Завдання 6

Переробіть фільтрацію з лаб. 3 на Mongoose Query API та додайте сортування і пагінацію.

Вимоги:

		Ліщинський В. В.			ДУ «Житомирська політехніка». 24.121.05.000 - ПЗ	Арк.
		Петросян А. Р.				7
Змн.	Арк.	№ докум.	Підпис	Дата		

- Збережіть щонайменше 2 query-параметри фільтрації з лаб. 3, переробивши логіку з in-memory фільтрації на умови Mongoose-запиту — використовуйте оператори порівняння, регулярні вирази або перелік значень залежно від типу поля.
- Додайте сортування через query-параметр sort. Визначте конвенцію для позначення напрямку сортування та реалізуйте її послідовно
- Реалізуйте пагінацію з параметрами page та limit зі зрозумілими значеннями за замовчуванням. Підрахунок загальної кількості документів та вибірку виконуйте паралельно.
- Формат відповіді GET /api/items змінюється: замість плоского масиву — об'єкт із полями data та pagination (поточна сторінка, ліміт, загальна кількість, кількість сторінок).
- Збережіть додатковий доменний маршрут з лаб. 3, адаптувавши його для Mongoose.

```

19 type CarQuery = {
20   brand?: string;
21   transmission?: 'manual' | 'automatic';
22   sort?: string;
23   page?: string;
24   limit?: string;
25 };
26
27 router.get('/', async (req: Request<{}, {}, {}>, CarQuery>, res: Response, next: NextFunction) => {
28   try {
29     const { brand, transmission, sort, page, limit } = req.query;
30
31     const filters: {
32       brand?: string;
33       transmission?: 'manual' | 'automatic';
34     } = {};
35
36     if(brand !== undefined) {
37       filters.brand = brand;
38     }
39
40     if(transmission !== undefined) {
41       filters.transmission = transmission;

```

Рис. 4.8 – Додано фільтрацію і пагінацію

Завдання 7

Перепишіть тести для роботи з реальною MongoDB у пам'яті замість in-memory Map

Вимоги:

- Встановіть mongodb-memory-server як dev-залежність.
- Створіть tests/setup.ts з допоміжними функціями запуску і зупинки тестової БД. • У кожному тестовому файлі використовуйте хуки beforeAll/afterAll для підключення та afterEach для очищення колекції між тестами.

		Ліщинський В. В.			ДУ «Житомирська політехніка». 24.121.05.000 - ПЗ	Арк.
		Петросян А. Р.				8
Змн.	Арк.	№ докум.	Підпис	Дата		

- Напишіть unit-тести моделі (в окремому блоці describe) — перевірте дефолти, автоматичні дати, правила валідації, віртуальні властивості тощо.
- Перепишіть інтеграційні тести з лаб. 3 для роботи з MongoDB — покрийте всі маршрути, включаючи граничні випадки специфічні для MongoDB.
- Фінальна кількість тестів — не менша за лаб. 3.
- Спробуйте досягти 100% покриття тестами коду, що стосується логіки проекту

```

tests > TS setup.ts > clearTestDB
1  import mongoose from 'mongoose';
2  import { MongoMemoryServer } from 'mongodb-memory-server';
3
4  let mongoServer: MongoMemoryServer;
5
6  export async function connectTestDB() {
7    mongoServer = await MongoMemoryServer.create();
8    const uri = mongoServer.getUri();
9    await mongoose.connect(uri);
10 }
11
12 export async function disconnectTestDB() {
13   await mongoose.connection.close();
14
15   if (mongoServer) {
16     await mongoServer.stop();
17   }
18 }
19
20 export async function clearTestDB() {
21   const collections = mongoose.connection.collections;
22
23   for (const key of Object.keys(collections)) {
24     const collection = collections[key];
25
26     await collection.deleteMany({});
27   }
28 }

```

Рис. 4.9 – setup.ts

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	100	96.96	100	100	
src	100	100	100	100	
app.ts	100	100	100	100	
src/middleware	100	100	100	100	
errorHandler.ts	100	100	100	100	
validate.ts	100	100	100	100	
src/models	100	100	100	100	
car.model.ts	100	100	100	100	
src/routes	100	100	100	100	
car.ts	100	100	100	100	
src/schemas	100	100	100	100	
car.schema.ts	100	100	100	100	
src/storage	100	100	100	100	
car.ts	100	100	100	100	
tests	100	50	100	100	
setup.ts	100	50	100	100	15-26
Test Suites: 1 passed, 1 total Tests: 29 passed, 29 total Snapshots: 0 total Time: 2.227 s Ran all test suites.					

Рис. 4.10 - Тести

Усі тести проекту проходять на 100%

Загальні вимоги

1. Весь код написано з використанням TypeScript.
2. Zod-схеми залишаються для HTTP-валідації; Mongoose-схема забезпечує цілісність даних у БД — обидва шари обов'язкові.
3. Правильні HTTP-статус-коди: 200 (GET, PATCH), 201 (POST), 204 (DELETE), 400 (невалідні дані / невалідний ID), 404 (не знайдено), 409 (конфлікт), 500 (несподівана помилка).
4. Обробка специфічних помилок MongoDB.
5. MongoDB credentials зберігаються у .env, файл .env додано до .gitignore.
6. Тести запускаються командою npm test.
7. Завантажте код на GitLab та надайте доступ викладачу.

Висновок

На даній лабораторній роботі ми набули практичних навичок роботи з MongoDB та Mongoose шляхом міграції REST API з лабораторної роботи №3 з in-memory сховища на повноцінну базу даних